

From Statistics to Machine Learning

Juan Pablo Lewinger

A tale of two questions

Consider the dataset — body fat percentage and body measurements in 252 men.

- **Question 1:** Does abdominal circumference affect body fat percentage, and by how much?
- **Question 2:** Given this patient's abdominal circumference, what is their body fat percentage?

Same data. Completely different goals.

- Question 1 is **statistics**: estimate an effect, test a hypothesis, interpret a coefficient
- Question 2 is **machine learning**: predict an outcome as accurately as possible in new individuals

The BodyFat dataset

Goal: predict body fat % (expensive: underwater weighing) from body circumferences (cheap: tape measure)

```
## Density BodyFat Age Weight Height Neck Chest Abdomen Hip Thigh Knee Ankle
## 1 1.0708 12.3 23 154.25 67.75 36.2 93.1 85.2 94.5 59.0 37.3 21.9
## 2 1.0853 6.1 22 173.25 72.25 38.5 93.6 83.0 98.7 58.7 37.3 23.4
## 3 1.0414 25.3 22 154.00 66.25 34.0 95.8 87.9 99.2 59.6 38.9 24.0
## 4 1.0751 10.4 26 184.75 72.25 37.4 101.8 86.4 101.2 60.1 37.3 22.8
## 5 1.0340 28.7 24 184.25 71.25 34.4 97.3 100.0 101.9 63.2 42.2 24.0
## 6 1.0502 20.9 24 210.25 74.75 39.0 104.5 94.4 107.8 66.0 42.0 25.6
## Biceps Forearm Wrist
## 1 32.0 27.4 17.1
## 2 30.5 28.9 18.2
## 3 28.8 25.2 16.6
## 4 32.4 29.4 18.2
## 5 32.2 27.7 17.7
## 6 35.7 30.6 18.8
```

Sample: 252 adult men, ages 22–81, mostly white, community-dwelling, United States, 1970s–80s.

What is machine learning?

The core idea: **teach a computer to learn from examples**, rather than programming rules by hand

- Supply the computer a set of examples with features/inputs and outcomes/labels
- The computer *learns* a function $\hat{y} = f(\mathbf{x})$ that maps inputs to outputs
- Goal: f should predict well on **new, unseen** examples

How does the computer learn f ? It searches over a family of possible functions and picks the one that best fits the training examples — e.g., minimizing prediction error. Different ML methods define different families of functions and different ways to search.

Three flavors of machine learning

- **Supervised learning:** labeled data, predict an outcome ← *our focus today*
 - Examples: predicting body fat %, diagnosing cancer from images, forecasting readmission
- **Unsupervised learning:** no labels, find structure (clusters, patterns)
 - Example: grouping patients by similar gene expression profiles
- **Reinforcement learning:** learn by reward and penalty
 - Example: training a robot or game-playing agent

Why ML? A biomedical motivation

ML is transforming prediction problems in medicine and public health:

- **Sepsis prediction:** a model detects sepsis risk 6 hours before clinical recognition — using only routine vitals and lab values from the EHR

Sepsis: a life-threatening overreaction of the body's immune response to infection — the body begins attacking its own tissues. Time-critical: every hour of delayed treatment increases mortality.

- **Diabetic retinopathy:** deep learning detects diabetic eye disease from fundus photographs with accuracy matching ophthalmologists

Diabetic retinopathy: damage to the blood vessels in the retina caused by chronically high blood sugar in diabetic patients. A leading cause of blindness, but largely preventable if caught early.

- **Hospital readmission:** models predict 30-day readmission risk from clinical records, enabling early intervention
- **Genomic risk scores:** models trained on thousands of genetic variants predict disease risk in individuals

By the end of today, you will understand the core ideas that make these models work.

Prediction vs. inference: a crucial distinction

	Statistical Inference	Machine Learning / Prediction
Goal	Estimate effects, test hypotheses	Predict outcomes in new individuals
Question	Does BMI causally affect diabetes risk?	What is this patient's 5-year diabetes risk?
We care about	Coefficients, p-values, confidence intervals	Accuracy on held-out data
Interpretability	Essential	Sometimes critical, sometimes irrelevant
Model complexity	Parsimony valued	Chosen to maximize predictive accuracy

You already know machine learning

Linear regression: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p$

Logistic regression: $\log \frac{p}{1-p} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p$

In **statistics**, after fitting we ask:

- Is $\hat{\beta}_j$ significantly different from zero?
- What is the confidence interval for $\hat{\beta}_j$?
- Does smoking *cause* higher body fat?

In **machine learning**, after fitting we ask:

- How accurately does this model predict body fat in *new* individuals?
- Can we do better with a different model?

Same model. Different questions asked of it. Today we use linear regression learn core ideas of ML.

Discussion question

Suppose we fit a linear regression predicting body fat % from abdominal circumference.

How would you know if the model is any good?

Two problems

Today we will encounter two distinct problems.

Problem 1 — Evaluation bias

Training performance always overestimates how well a model generalizes — *for any model*, regardless of complexity. The remedy is a held-out **test set**.

Problem 2 — Overfitting

A model that is too complex fits the noise in the training data, not just the signal. This can makes it predict new patients worse. The remedy is **controlling complexity**.

We tackle Problem 1 first.

Performance metrics

Baseline model: with no predictors, the best constant prediction for everyone is the mean $\bar{y}$.

Its total error: $TSS = \sum_{i=1}^n (y_i - \bar{y})^2$ (Total Sum of Squares)

Your regression model does better (hopefully). Its error: $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ (Residual Sum of Squares)

R^2 measures how much better your model is than the mean-only baseline:

$$R^2 = 1 - \frac{RSS}{TSS}$$

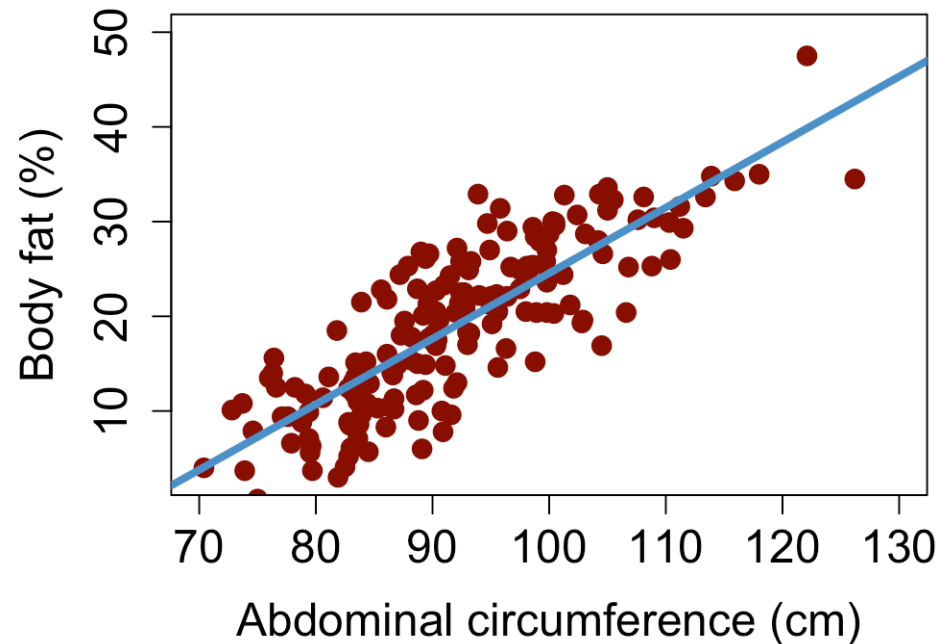
- $R^2 = 1$: perfect prediction — every $\hat{y}_i = y_i$
- $R^2 = 0$: no better than just predicting the mean for everyone
- **In the test set, R^2 can be negative** — the model does *worse* than predicting the mean. A sure sign of a badly overfit model.

Interpretation: “My model explains **X%** of the variation in body fat that the mean-only model couldn’t explain.”

Also define RMSE: $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ — on the same scale as the outcome (% body fat).

Problem 1: the trap of evaluating on training data

Natural instinct: fit model $\rightarrow$ compute $R^2 \rightarrow$ if R^2 is high, we're done.



$R^2_{train} = 0.677$, $RMSE_{train} = 4.87\%$ body fat — looks good! But what does this actually tell us?

What training R^2 really measures

The model was chosen specifically to minimize error on the training data.

Analogy: a student who memorizes the practice exam

- Scores 100% on the practice exam
- Does that tell you anything about how they'll do on the real exam?

The model has “seen” every training observation and it adapts to it.

Key insight: a model's performance on the data used to build it is *not* a reliable measure of how well it will generalize to new patients — even if the model is perfectly well-specified.

The solution: hold out data

Reserve part of the data **before fitting any model**. **Never use for fitting**.

The test set is a stand-in for (a random sample of) future patients the model will encounter in the real world. Evaluating on it gives an honest estimate of generalization.

Analogy: the real exam, not the practice exam. The model studied from the training data; the test set is the exam it has never seen.

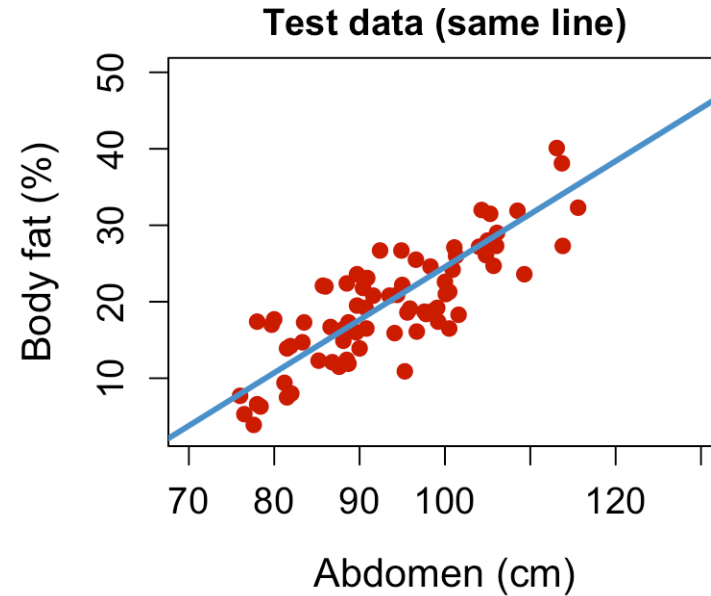
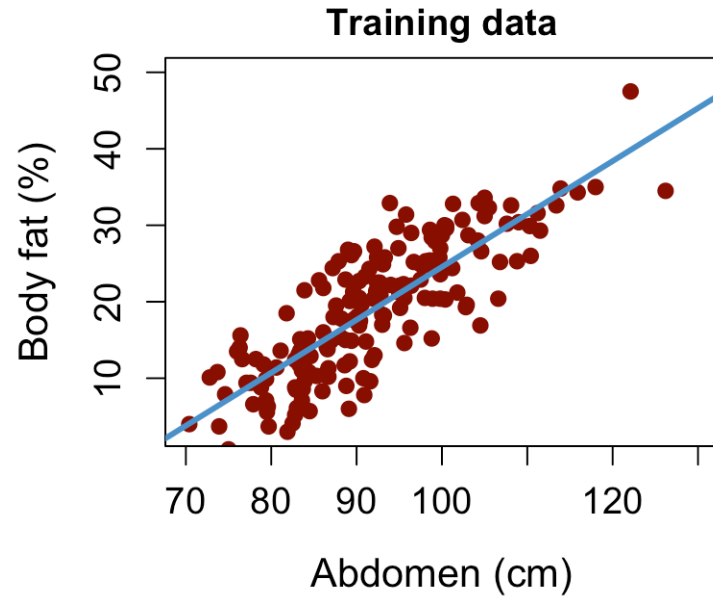
The train/test split in R

```
set.seed(301)                # for reproducibility
n <- nrow(bodyfat)
trainset <- sample(1:n, floor(0.7 * n))  # random 70% for training

bodyfat_train <- bodyfat[trainset, ]      # 70% – fit model here
bodyfat_test  <- bodyfat[-trainset, ]     # 30% – evaluate here
```

- 70% train / 30% test is a common starting point
- `set.seed()` ensures the same split every time (reproducibility matters!)
- The split must be done **before any modeling decisions** — no peeking

Fitting on training data, evaluating on test data



	Train	Test
R^2	0.677	0.559
RMSE	4.87%	5.01%

$R^2_{test} < R^2_{train}$ — this gap is the **evaluation bias**.

After evaluation: which model do you use?

OK — we used the test data to estimate prediction error.

But what model should we use to predict body fat in **new patients**?

A. The model fitted on training data only?

B. The model refitted on the combined training + test data?

Answer

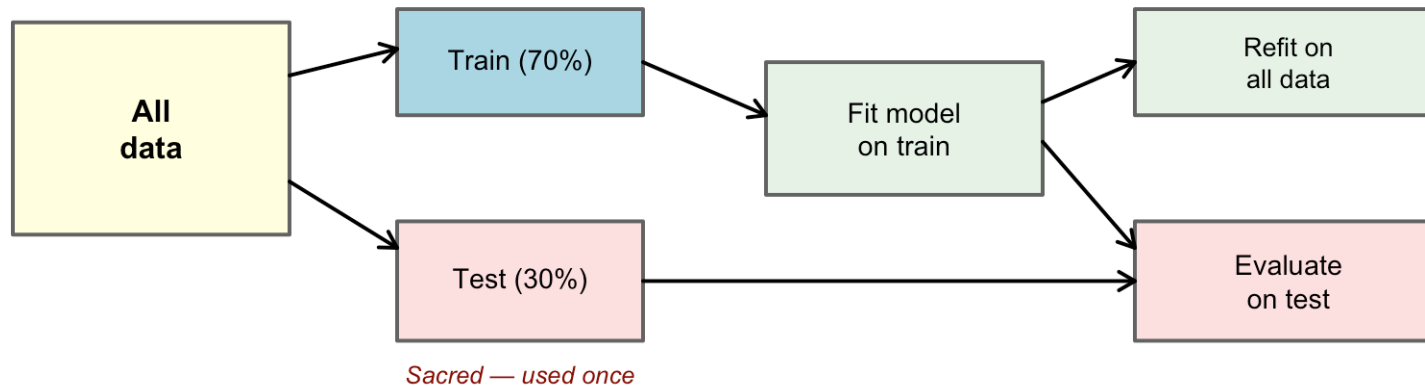
B. Refit on all available data.

The test set was reserved for evaluation — that job is now done.

There is no longer any reason to withhold it. Fitting on more data generally gives a better model.

The final model is fitted on the full dataset; its expected performance in new patients is the test R^2 we already estimated.

The ML workflow — so far



Classical stats: all data → fit → interpret coefficients → done.

ML: split → train → **evaluate honestly on held-out test set** → refit on all data → deploy.

Problem 2: overfitting

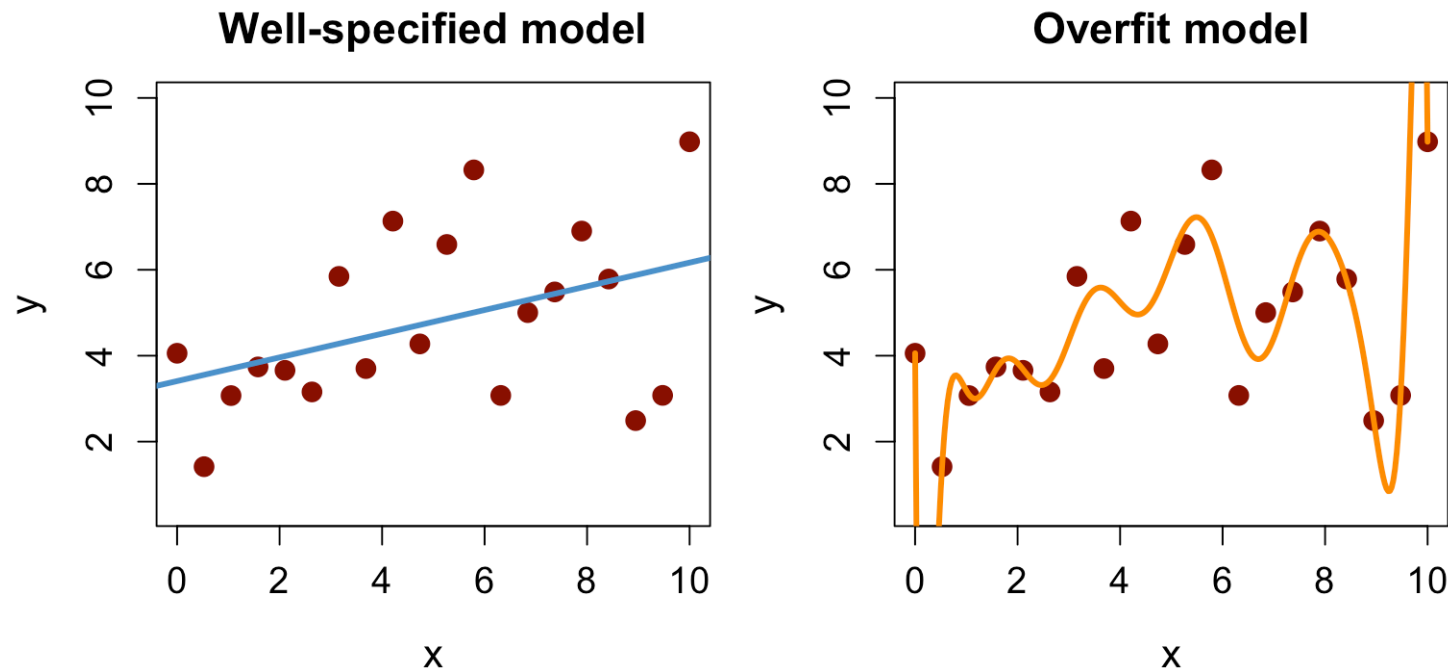
Overfitting: a model that is too complex fits the specific noise in the training data, not the underlying signal. It can then perform *worse* on new data.

A model can be:

- **Underfit:** too simple, misses real patterns (high bias)
- **Well-specified:** right complexity for the data
- **Overfit:** too complex, fits noise (high variance)

Overfitting: the concept

A model can be **too flexible** — it fits the noise in the training data, not the underlying signal.

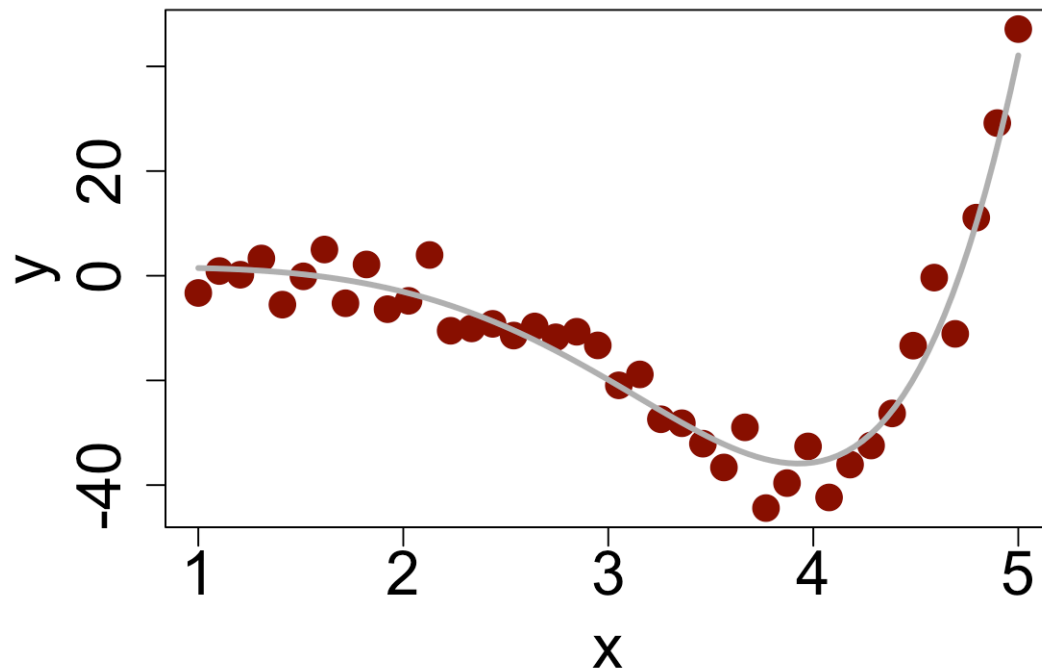


Which model would you trust for a **new patient**? The overfit model has low training error — but will fail on new data.

Overfitting: a simulation

Real data has an unknown true relationship — we never know if we're overfitting or underfitting. With simulated data we can cheat: the **true underlying function** (gray curve) is known. This lets us see clearly when a model has found the signal vs. chased the noise.

- $n = 40$ training observations
- True relationship: $y = e^x \cos(x) + \epsilon$, $\epsilon \sim N(0, \sigma^2)$, $\sigma = 5$



Fitting polynomials of increasing complexity

We fit polynomial regression models of increasing degree p — each one more flexible than the last:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

$$\vdots$$

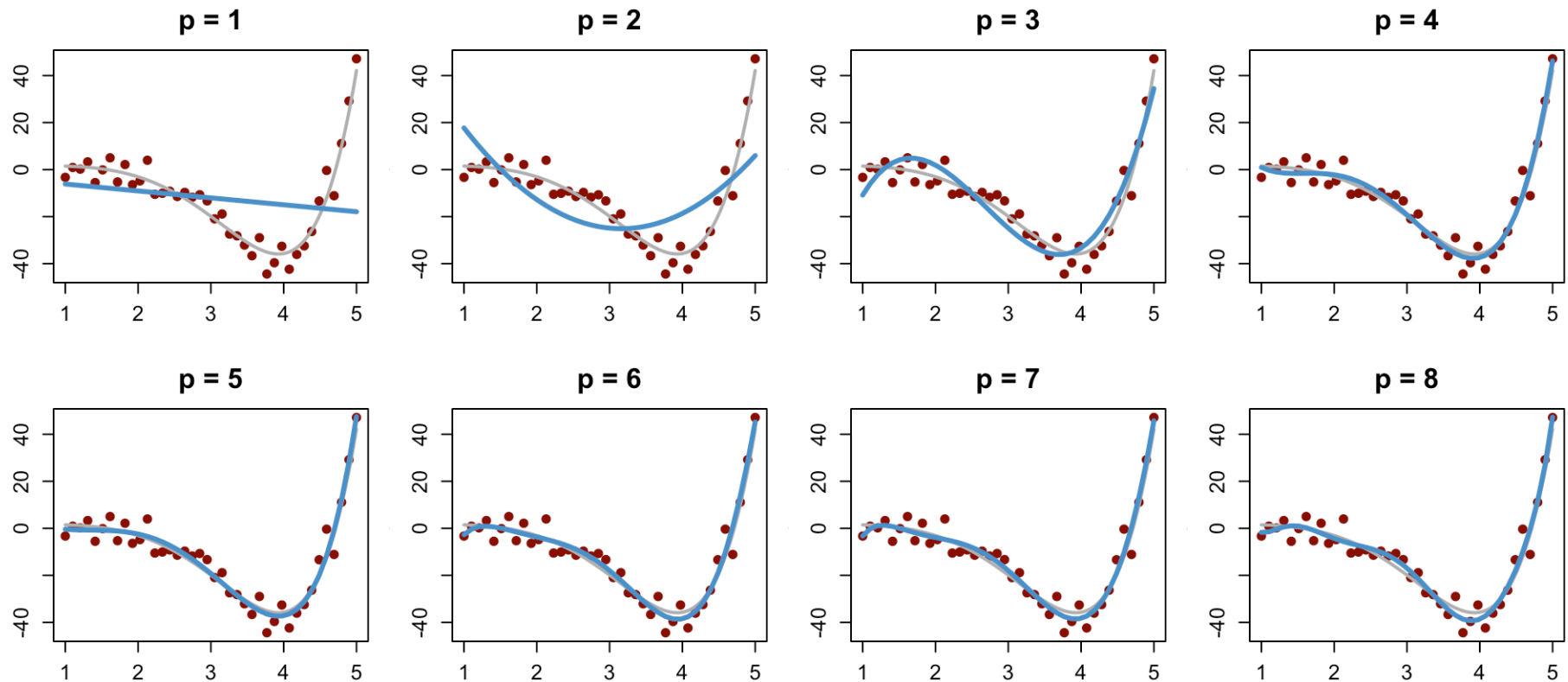
$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_{25} X^{25} + \epsilon$$

Fitting polynomials of increasing complexity

```
fit1 = lm(y ~ x, data = train)
fit2 = lm(y ~ x + I(x^2), data = train)
fit3 = lm(y ~ x + I(x^2) + I(x^3), data = train)
fit4 = lm(y ~ x + I(x^2) + I(x^3) + I(x^4), data = train)
fit5 = lm(y ~ x + I(x^2) + I(x^3) + I(x^4) + I(x^5), data = train)
fit6 = lm(y ~ x + I(x^2) + I(x^3) + I(x^4) + I(x^5) + I(x^6), data = train)
fit7 = lm(y ~ x + I(x^2) + I(x^3) + I(x^4) + I(x^5) + I(x^6) + I(x^7), data = train)
fit8 = lm(y ~ poly(x, 8), data = train)
fit9 = lm(y ~ poly(x, 9), data = train)
fit15 = lm(y ~ poly(x, 15), data = train)
fit20 = lm(y ~ poly(x, 20), data = train)
fit25 = lm(y ~ poly(x, 25), data = train)
```

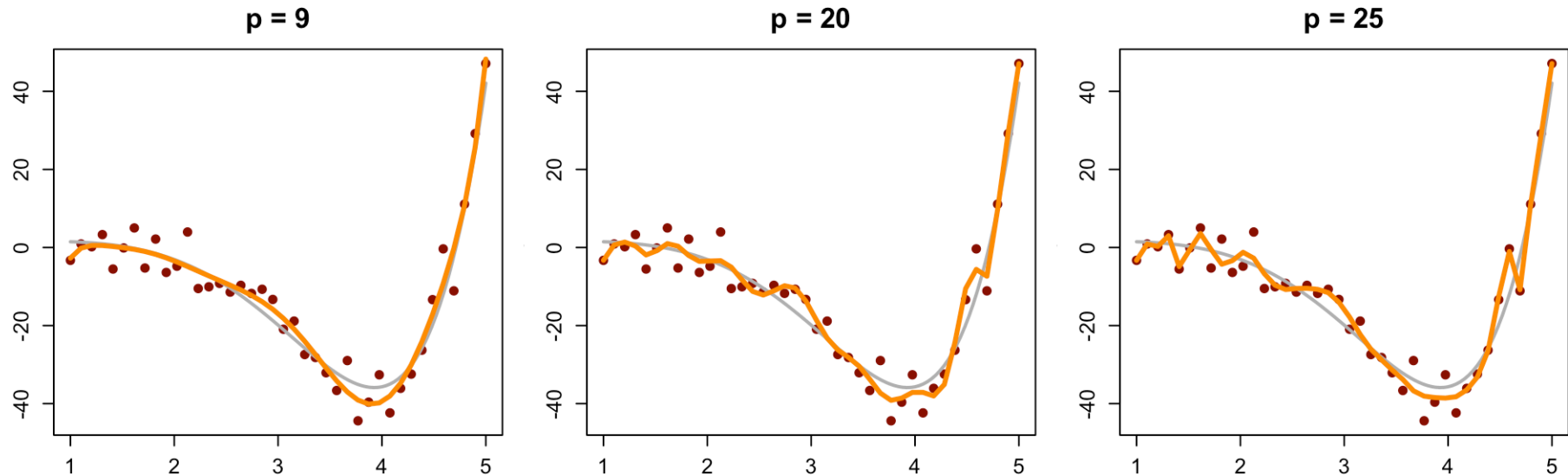
Using poly() is more compact and numerically better!

How the fits look: low complexity (training data)



Gray curve = true underlying function.

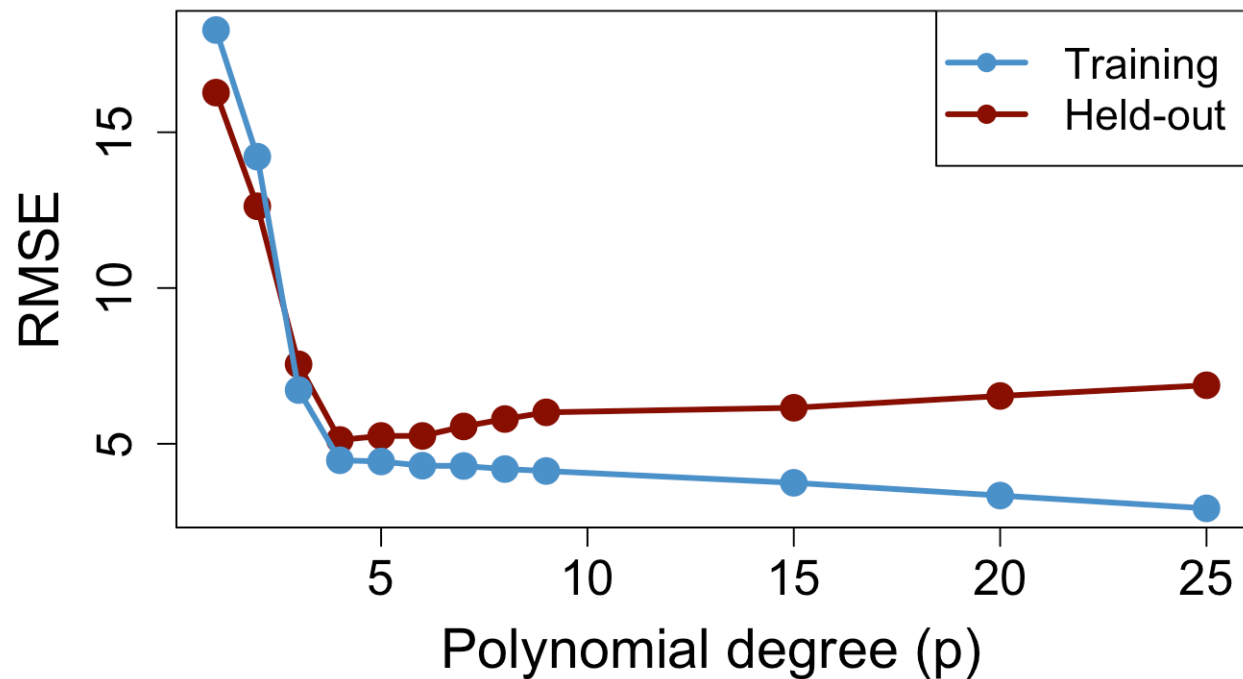
How the fits look: high complexity (training data)



High-degree fits close to every training point — they have memorized the noise, not learned the signal.

Training vs. held-out error by complexity

Training looks great for high-degree models — but how do they do on data they haven't seen?



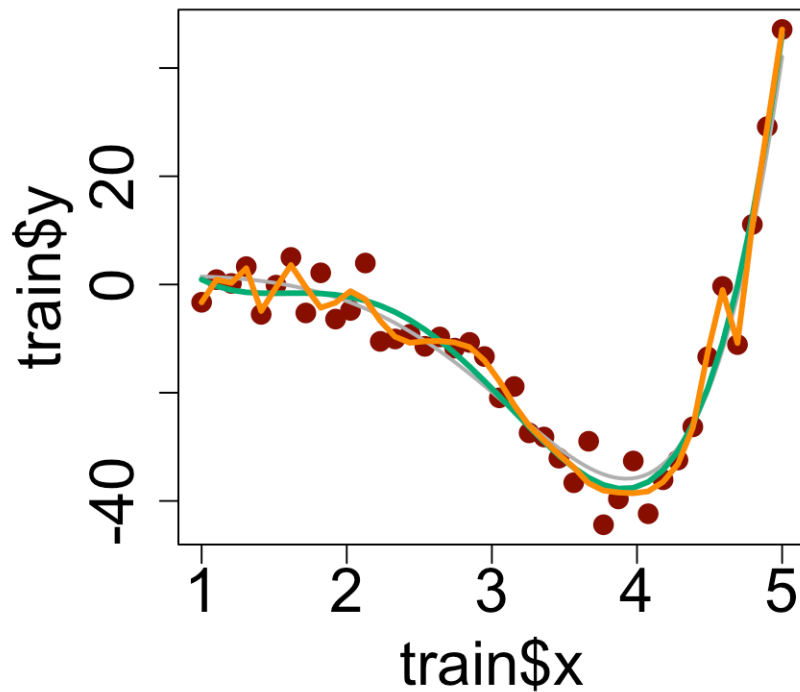
Training vs. held-out error by complexity

	p=1	p=2	p=3	p=4	p=5	p=6	p=7	p=8	p=9	p=15	p=20	p=25
## Training	18.28	14.22	6.73	4.47	4.43	4.30	4.29	4.19	4.13	3.75	3.34	2.93
## Held-out	16.27	12.63	7.55	5.12	5.25	5.25	5.56	5.79	6.01	6.16	6.54	6.88

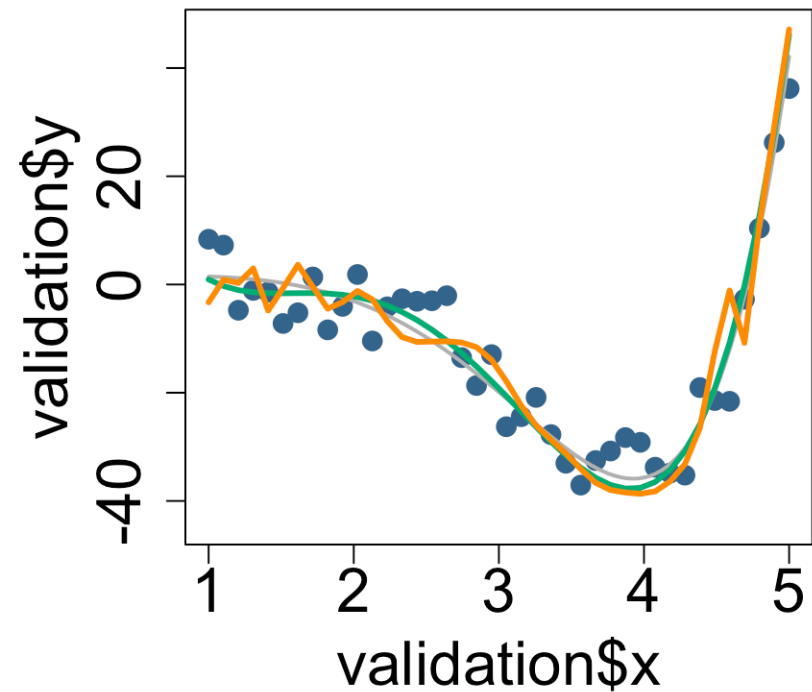
- Training RMSE decreases monotonically — it always rewards more complexity
- Held-out RMSE has a clear minimum around $p = 4$ — this is the model we would select
- Models with $p \geq 9$ have high held-out error despite low training error
- $p = 5$ or $p = 6$ give very similar held-out RMSE — with finite data there is uncertainty in the exact optimum

Overfitting

Training - $p = 4$ and $p = 25$



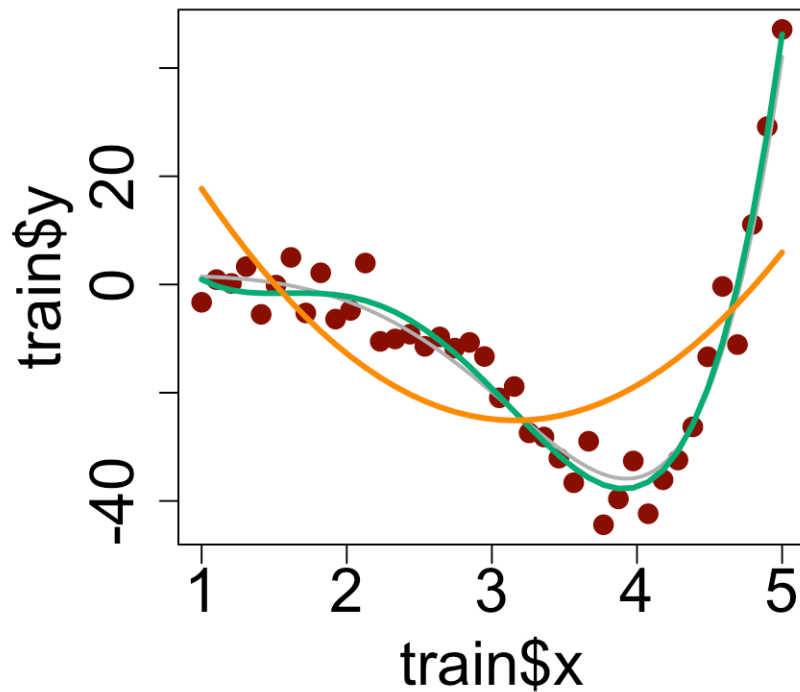
held-out - $p = 4$ and $p = 25$



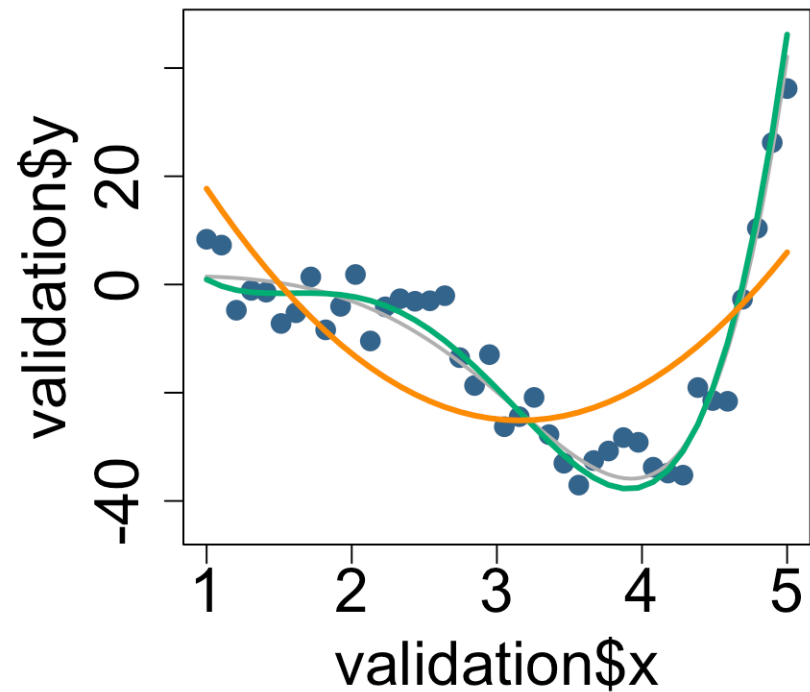
- Overfitting ($p = 25$, orange): looks excellent on training data — collapses on held-out data
- Green ($p = 4$): generalizes well — close to the true function on both sets

Underfitting

Training - $p = 2$ and $p = 4$



Validation - $p = 2$ and $p = 4$



- Underfitting ($p = 2$, orange): too rigid to follow the non-linear true curve — performs poorly on both sets
- Green ($p = 4$): right complexity — captures the underlying trend without chasing noise

Overfitting and underfitting: summary

Balancing complexity is a central challenge in ML.

- **Underfitting:** model too simple — misses the signal. Performs poorly on both training and validation data. Fix: consider more flexible models.
- **Overfitting:** model too complex — fits the noise. Looks great on training data, fails on validation. Fix: reduce complexity or use more data.
- In practice we don't know the right complexity beforehand — we need to find it from the data. How to do this properly is what we turn to next.

Model **complexity** increases with: number of features, degree of polynomial terms, number of interactions, and type of model (tree-based and neural network models are far more flexible than linear regression).

Model selection: the general problem

The simulation illustrated the challenge with a single predictor and polynomial degree as the complexity knob. In practice, the choices are more varied — we may want to try:

- Different sets of predictors (abdomen only vs. abdomen + age + weight + ...)
- Interaction terms or higher-order (polynomial) terms
- Entirely different families of models (not just linear regression)

Suppose you have two candidates:

- **Model A:** abdomen circumference only
- **Model B:** abdomen + age + weight + neck + chest circumference

You fit both on the training data, evaluate both on the test set — Model B wins.

So you report Model B's test R^2 and call it a day. **Is this valid?**

The problem with using the test set to choose

When you use the test set to **select** the better model, it is no longer a fair evaluation.

You have effectively trained on the test set — indirectly.

- You tried Model A $\rightarrow$ test $R^2 = 0.68$
- You tried Model B $\rightarrow$ test $R^2 = 0.73$
- You chose Model B because it won on the test set

The test R^2 of the winner is now **optimistic** — it benefited from a selection process.

The test set can only be used **once**, for **one** purpose — final evaluation of a model.

The solution: a three-way split

Introduce a **validation set** — a second held-out set, separate from the test set.

Set	Size	Purpose
Training	~60%	Fit the model
Validation	~20%	Evaluate candidates; choose the best one
Test	~20%	Final honest evaluation — one time only

If you try a very large number of models on the validation set, the winner may still overfit to it — the same selection bias applies.

The test set remains a **locked box** — opened only after all modeling decisions are final.

The cost: less data for everyone

With a 60/20/20 split and $n = 252$:

- Training: ~150 observations — fewer observations to fit the model
- Validation: ~50 observations — noisy model selection
- Test: ~50 observations — noisy performance estimate

Using only 60% of data for fitting is costly — more data almost always means a better model.

This tension is real and painful in biomedical research, where datasets are often small.

Can we do better? Yes — **cross-validation** gives a smarter way to estimate generalization performance without sacrificing as much data.

Discussion question

A colleague fits **10 different models**, evaluates each one on the test set, picks the best, and reports that test performance.

What is wrong with this approach?

Ethics thread: biased evaluation at the population level

A model trained and evaluated on the BodyFat dataset may look excellent within that sample:

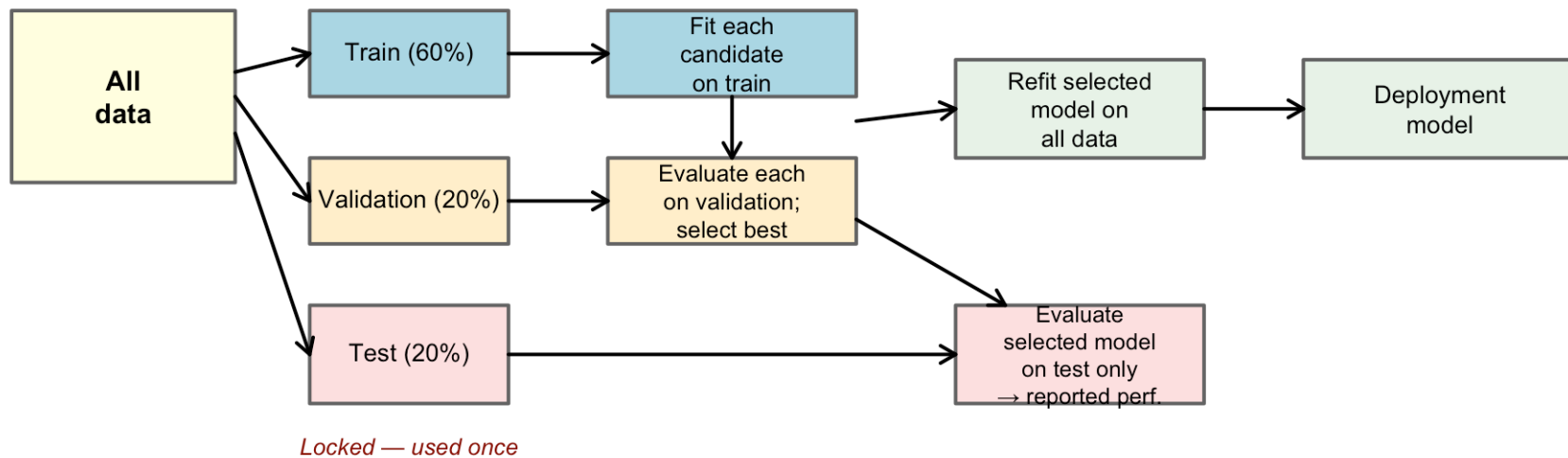
- 252 men, mostly white, middle-aged, community-dwelling, United States, 1970s–80s

But what happens when deployed on women? Younger or older patients? Different ethnic backgrounds?

Evaluation on held-out data from the same biased sample does not catch this. The test set only tells you how well you generalize *within* the population your data came from — not beyond it.

This is one of the most consequential failure modes in clinical ML. Models trained predominantly on one group have been shown to perform worse — sometimes dangerously so — on others.

The complete ML workflow



Coming up: the lab

In the lab you will:

1. See evaluation bias firsthand — a well-specified model shows a train/test gap
2. Compare a simple and a richer model on the test set
3. See overfitting in action as polynomial terms push test R^2 negative

A script will be provided — focus on the **insight**, not the R syntax.

Coming up: Lecture 2

- **Cross-validation:** a smarter way to estimate generalization when data is scarce — replaces the validation set without sacrificing data
- **Decision trees:** a completely different kind of model — intuitive, visual, and naturally non-linear
- **Random forests:** fixing trees' overfitting problem — wisdom of crowds, applied to models

Key takeaways

- ML and statistics share many of the same models but ask different questions: **prediction vs. inference**
- **Evaluation bias** is always present: training performance overestimates generalization for any model — a held-out test set is the remedy
- **Overfitting** too much complexity causes a model to fit noise and perform worse on new data — controlling complexity is the remedy
- **Model selection** requires a validation set — using the test set to choose between models defeats its purpose
- In small biomedical datasets, the 3-way split is costly — cross-validation (Lecture 2) offers a better solution